

1.1 Starting Hive & Database Configuration

```
hive -- Launch Hive CLI
beeline -u jdbc:hive2://localhost:10000 -- Launch via Beeline
(HiveServer2)
hive --hiveconf hive.root.logger=DEBUG,console -- Launch with debug logging

set hive.cli.print.current.db=true; -- Show current database in
prompt
set hive.cli.print.header=true; -- Show column headers in
output
set hive.exec.mode.local.auto=true; -- Enable local mode for small
data
set hive.execution.engine=tez; -- Switch engine: mr / tez /
spark
```

Database Operations

. Create, use, describe, and drop a database.

Answer:

```
CREATE DATABASE IF NOT EXISTS rahul
COMMENT "Database for training practicals"
LOCATION "/user/hive/warehouse/rahul.db"
WITH DBPROPERTIES ("creator"="instructor", "date"="2025");

SHOW DATABASES; -- List all databases
SHOW DATABASES LIKE "train*"; -- Filter by pattern
USE training_db; -- Switch to database
DESCRIBE DATABASE EXTENDED rahul; -- Full database details
DROP DATABASE IF EXISTS training_db CASCADE; -- Drop (CASCADE removes
tables)

CREATE TABLE nyse_m (
  stk_date DATE,
  symbol STRING,
  open DOUBLE,
  close DOUBLE,
  low DOUBLE,
  high DOUBLE,
```

```

        volume    DOUBLE
    )
    ROW FORMAT DELIMITED
    FIELDS TERMINATED BY ','
    STORED AS TEXTFILE
    TBLPROPERTIES ("skip.header.line.count"="1");

LOAD DATA INPATH '/user/cddacmaylab014/data/nyse.csv'
INTO TABLE nyse_m; # hadoop ka path jayega

LOAD DATA LOCAL INPATH '/home/cddacmaylab014/nyse.csv'
INTO TABLE nyse_managed; # client ka path cjahiye

```

Create an External Table over existing HDFS data.

```

CREATE EXTERNAL TABLE nyse_ext (
    stk_date DATE,
    symbol    STRING,
    open      DOUBLE,
    close     DOUBLE,
    low       DOUBLE,
    high      DOUBLE,
    volume    DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/cddacmaylab014/data'
TBLPROPERTIES ("skip.header.line.count"="1");

```

2.3 Table Introspection Commands

Q3. Inspect a table's structure, location, and properties.

Answer:

```

DESCRIBE nyse;                -- Column names and types
DESCRIBE FORMATTED nyse;      -- Full details: location, format, stats

```

```

DESCRIBE EXTENDED nyse;      -- Raw metadata (serialized form)
SHOW TABLES;               -- List tables in current DB
SHOW TABLES LIKE "nyse*";  -- Filter tables by pattern
SHOW CREATE TABLE nyse;    -- Show the DDL used to create the table
SHOW TBLPROPERTIES nyse;    -- Show table properties

```

Note: *DESCRIBE FORMATTED* is most useful — it shows storage format, SerDe info, partition info, and HDFS location.

2.4 ALTER TABLE — Modifying Table Structure

Q4. Add, rename, and drop columns; change table properties.

Answer:

```

-- Add a new column
ALTER TABLE nyse ADD COLUMNS (sector STRING COMMENT "Industry sector");

-- Rename a column (CHANGE also moves column)
ALTER TABLE nyse CHANGE open open_price DOUBLE;

-- Replace all columns (re-define entire schema)
ALTER TABLE nyse REPLACE COLUMNS (stk_date STRING, symbol STRING, price
DOUBLE);

-- Rename the table
ALTER TABLE nyse RENAME TO nyse_stock;

-- Set/change table properties
ALTER TABLE nyse SET TBLPROPERTIES ("skip.header.line.count"="1");
ALTER TABLE nyse SET TBLPROPERTIES ("comment"="NYSE stock prices");

```

Create a partitioned table by year.

Answer:

```

CREATE TABLE nyse_partitioned (
    stk_date STRING,
    symbol STRING,
    open FLOAT,
    close FLOAT,
    low FLOAT,
    high FLOAT,
    volume FLOAT
)
PARTITIONED BY (year STRING)

```

```
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t';
```

PARTITIONED BY (year STRING)

This tells Hive that the data will be divided (partitioned) by the year column.

The column year is not part of the main data — it's used as a directory name in HDFS.

ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'

The data fields in the file are separated by tabs (t).

Because i dont have different csv for each year

Instead, we'll let **Hive create partitions automatically** using **dynamic partitioning**.

Or else you have a direct file static partition can be done

```
-LOAD DATA INPATH '/user/data/nyse_2016.csv' INTO TABLE nyse_partitioned
PARTITION (year='2016'); // same command for each year
```

```
SET hive.exec.dynamic.partition=true;
```

```
SET hive.exec.dynamic.partition.mode=nonstrict;
```

To load data:

```
INSERT OVERWRITE TABLE nyse_partitioned
PARTITION (year)
SELECT
    stk_date,
    symbol,
    open,
    close,
    low,
    high,
    volume,
    year(stk_date) AS year
FROM nyse_m;
```

Time comparison :

A. On the non-partitioned table (nyse)

```
SELECT COUNT(*)  
FROM nyse_m  
WHERE year(stk_date) = 2016;
```

```
SELECT symbol, AVG(close) AS avg_close_2016 FROM nyse WHERE year(stk_date) =  
2016 GROUP BY symbol;
```

B. On the partitioned table (nyse_partitioned)

```
SELECT COUNT(*)  
FROM nyse_partitioned  
WHERE year = '2016';
```

```
SELECT symbol, AVG(close) AS avg_close_2016 FROM nyse_partitioned WHERE  
year(stk_date) = 2016 GROUP BY symbol;
```

Q9. Show all partitions in a table.

Answer:

```
SHOW PARTITIONS nyse_partitioned;
```

Section 4: What is Bucketing in Hive?

Bucketing is a way to further divide data inside each partition (or table) into smaller, fixed-size files called buckets — based on the hash value of a column.

Think of it like this:

Partitioning splits data by a category (e.g., year = 2016, 2017...)

Bucketing splits data within each partition into equal-sized buckets based on a column's hash value (e.g., by symbol).

Q12. Create a bucketed table with 4 buckets.

Answer:

```
CREATE TABLE nyse_bucketed (  
    stk_date STRING,  
    symbol STRING,  
    open FLOAT,  
    close FLOAT,  
    low FLOAT,  
    high FLOAT,  
    volume FLOAT  
)  
CLUSTERED BY (symbol) INTO 4 BUCKETS  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t';
```

How Hive Decides the Bucket

Hive computes:

$$\text{bucket_number} = \text{hash}(\text{symbol}) \% \text{number_of_buckets}$$

So if `symbol='AAPL'` hashes to 2, it goes into bucket 2.

Thus, data with the **same symbol value always goes to the same bucket** — even if you reprocess later (for join optimization).

Example

symbol	Java hashCode	After mod 4	Goes to bucket
AAPL	63281924	0	bucket_0
MSFT	77464855	3	bucket_3
WLTW	88742467	3	bucket_3
TATA	2656907	3	bucket_3
INFY	23456781	1	bucket_1

/nyse_bucketed/000000_0 → contains AAPL

/nyse_bucketed/000001_0 → contains INFY

/nyse_bucketed/000003_0 → contains MSFT, WLTW, TATA

Loading Data into a Bucketed Table:

```
SET hive.enforce.bucketing=true;
```

```
INSERT OVERWRITE TABLE nyse_bucketed SELECT * FROM nyse;
```

Difference Between Partitioning and Bucketing

Feature	Partitioning	Bucketing
Basis	Column values (e.g., year=2016)	Hash of a column (e.g., hash(symbol) % 4)

Data location	Different folders per partition	Different files within same folder
Granularity	Coarse (macro-level split)	Fine (micro-level split)
Used for	Query pruning	Faster joins, balanced data
Example	PARTITIONED BY (year STRING)	CLUSTERED BY (symbol) INTO 4 BUCKETS

Q18. Create a managed table from an existing table.

Answer:

```
CREATE TABLE nyse_copy AS  
SELECT * FROM nyse;
```

Q19. Export Hive table data to HDFS.

Answer:

```
INSERT OVERWRITE DIRECTORY '/user/cdacoctnov012/db'  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t'  
SELECT * FROM nyse;
```

Q20. Drop a Hive table (and delete data).

Answer:

```
DROP TABLE nyse;
```


How TEXTFILE Stores It

A **TextFile** or **CSV** table will store it **row by row**:

```
1,Rahul,25,India
2,Meena,30,USA
3,Aditya,27,India
4,John,35,UK
5,Priya,22,India
```

Every query reads all fields — even if you just want country.

How ORC Stores It (Columnar Format)

In ORC, data is stored **column-wise** like this:

```
Column 1 (cust_id): [1, 2, 3, 4, 5]
Column 2 (name):   [Rahul, Meena, Aditya, John, Priya]
Column 3 (age):    [25, 30, 27, 35, 22]
Column 4 (country): [India, USA, India, UK, India]
```

Each **column** is stored **together**, not each row.

This allows compression algorithms (like ZLIB or Snappy) to compress similar data **very efficiently**.

When you query only `SELECT country FROM customers`, Hive reads only the country column blocks — skipping all others.

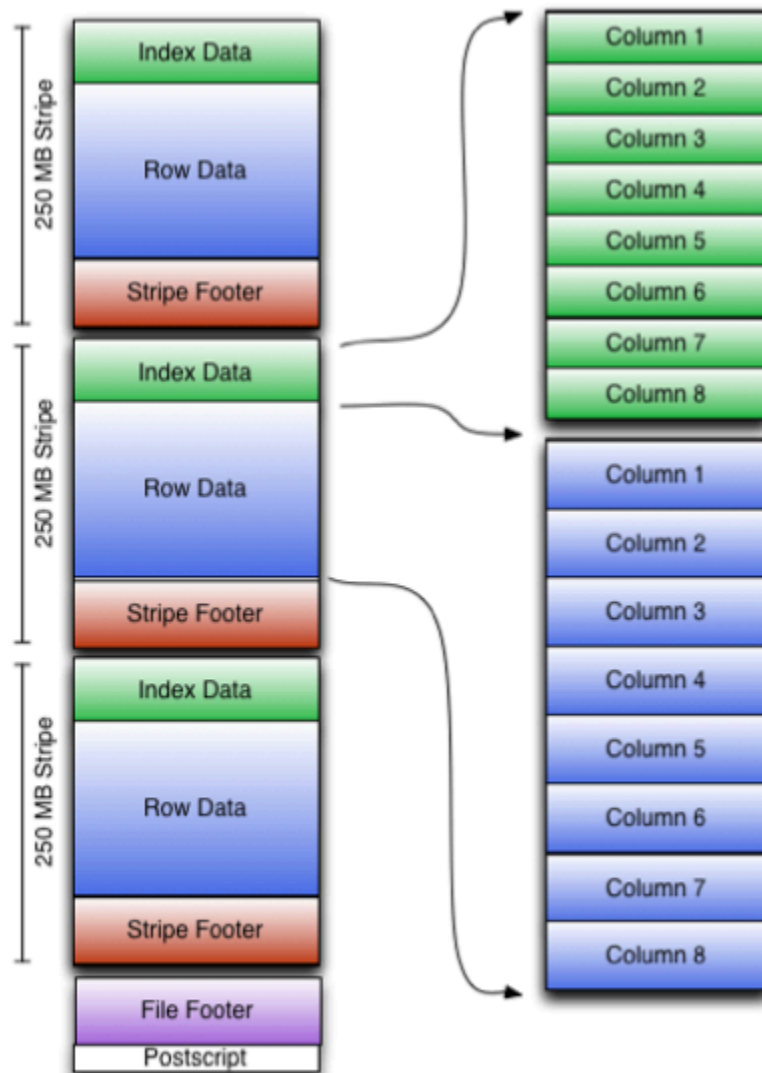
ORC File Internal Structure

ORC file is divided into **stripes**, and each stripe has:

1. **Index Data** — min, max, and row positions for quick filtering
2. **Row Data** — actual encoded & compressed column data
3. **Stripe Footer** — metadata for that stripe

And the file ends with:

- **File Footer** (metadata about stripes, column types)
- **Postscript** (compression info, file length, etc.)



Each **stripe** might hold, say, 250 MB of data (configurable).

Example of How It Optimizes a Query

Let's say you run:

```
SELECT cust_id, country FROM customers WHERE country = 'India';
```

Hive + ORC will:

- Use the **column index** to find which stripes have country = 'India'
- **Skip reading entire stripes** where country ≠ India
- Only decompress and read cust_id and country columns

A. Customer Table

```
CREATE TABLE customers (  
  cust_id INT,  
  fname STRING,  
  lname STRING,  
  age INT,  
  profession STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

B. Transaction Table

```
CREATE TABLE transactions (  
  txn_id STRING,  
  txn_date STRING,  
  cust_id INT,  
  amount FLOAT,  
  category STRING,  
  product STRING,  
  city STRING,  
  state STRING,  
  payment_mode STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

Load Data

```
LOAD DATA Local INPATH '/home/cddacmaylab014/custs.txt' INTO TABLE
customers;
LOAD DATA Local INPATH '/home/cddacmaylab014/txns1.txt' INTO TABLE transactions;
```

Section 3: Hive Data Types

Hive supports primitive, complex, and special data types. Understanding these is critical for correct schema design.

3.1 Primitive Types

Category	Data Type	Description / Example
Integer	TINYINT	1 byte: -128 to 127
Integer	SMALLINT	2 bytes: -32,768 to 32,767
Integer	INT	4 bytes: standard integer
Integer	BIGINT	8 bytes: large integers
Floating	FLOAT	4-byte single precision
Floating	DOUBLE	8-byte double precision (recommended)
Floating	DECIMAL(p,s)	Exact decimal — DECIMAL(10,2) for money
Text	STRING	Variable-length text (most common)
Text	VARCHAR(n)	Variable-length, max n chars
Text	CHAR(n)	Fixed-length n chars
Date/Time	DATE	YYYY-MM-DD only
Date/Time	TIMESTAMP	YYYY-MM-DD HH:MM:SS.nanoseconds
Boolean	BOOLEAN	TRUE or FALSE
Binary	BINARY	Byte array

3.2 Complex Types

Type	Syntax	Example Usage
ARRAY	ARRAY<data_type>	ARRAY<STRING> — e.g., ['AAPL','MSFT']
MAP	MAP<key_type, value_type>	MAP<STRING,DOUBLE> — e.g., {open':120.5}
STRUCT	STRUCT<name:type,...>	STRUCT<city:STRING, zip:INT>
UNIONTYPE	UNIONTYPE<t1,t2,...>	Holds one of several types (rare)

Section 4: File Formats & SerDe

Choosing the right file format dramatically affects storage efficiency and query performance.

Format	Stored As	Compression	Best For	Splittable
TextFile	TEXTFILE	None/GZIP	Raw CSV/TSV ingestion	Yes (line-based)
ORC	ORC	ZLIB/Snappy/ZSTD	Analytics, ACID, production	Yes
Parquet	PARQUET	Snappy/GZIP	Spark/Hive interop, columnar	Yes
Avro	AVRO	Snappy/Deflate	Schema evolution, Kafka data	Yes
SequenceFile	SEQUENCE FILE	BLOCK/RECORD	Legacy MapReduce pipelines	Yes
RCFile	RCFILE	ZLIB	Older columnar (ORC is better)	Yes

```
CREATE TABLE txns_orc
STORED AS ORC
AS SELECT * FROM transactions;
```

Parquet Table

```
CREATE TABLE txns_parquet  
STORED AS PARQUET  
AS SELECT * FROM transactions;
```

➡ Compare size:

```
hdfs dfs -du -h /user/hive/warehouse/rahul.db/txns_orc
```

```
hdfs dfs -du -h /user/hive/warehouse/rahul.db/txns_parquet
```

```
hdfs dfs -du -h /user/hive/warehouse/rahul.db/transactions
```

➡ Compare performance:

```
SELECT COUNT(*) FROM txns_orc;  
SELECT COUNT(*) FROM txns_parquet;
```

ORC usually smaller and faster in Hive; Parquet interoperable with Spark.

Module: ACID Tables & Hive UDFs (using customers & transactions tables)

Part 1: ACID & Transactional Tables

Concept Overview

Hive supports **ACID (Atomicity, Consistency, Isolation, Durability)** properties for **transactional tables** starting from Hive 0.14+.

They allow:

- INSERT, UPDATE, DELETE operations (like RDBMS)
- Small transactions with row-level locking

Pre-requisites

Make sure these are set before creating ACID tables:

SET hive.cli.print.current.db=true;

SET hive.cli.print.header=true;

SET hive.support.concurrency = true;

SET hive.txn.manager = org.apache.hadoop.hive.ql.lockmgr.DbTxnManager;

SET hive.enforce.bucketing = true;

Why Bucketing is Required in Transactional Tables

- **Hive ACID tables** support INSERT, UPDATE, DELETE — so Hive must **track each row** precisely.
- **Bucketing** ensures each row always goes into the **same file (bucket)** based on a hash of the key (e.g., cust_id).
- This makes **updates and deletes faster**, since Hive can find and modify only the affected bucket, not the whole table.
- **ORC + Bucketing** are **mandatory** for transactional tables because ORC stores row-level metadata and bucketing gives deterministic storage.

Delta Files in Hive ACID Tables

- When you **UPDATE** or **DELETE**, Hive doesn't rewrite the whole ORC file.
- It creates **delta files** (like delta_0002_0003) for each bucket to store only the changes.
- The **base** folder has the original data, and Hive **merges base + delta** files when you query — showing the latest data.
- Over time, many deltas are **compacted** into a new base file for better performance.

Create an ACID (Transactional) Table

We'll convert the existing customers table into a **transactional ORC table**.

```
CREATE TABLE customers_txn (  
  cust_id INT,  
  fname STRING,  
  lname STRING,  
  age INT,  
  profession STRING  
)  
CLUSTERED BY (cust_id) INTO 4 BUCKETS  
STORED AS ORC  
TBLPROPERTIES ('transactional'='true');
```

Load data:

```
INSERT INTO customers_txn SELECT * FROM customers;
```

Demonstrate Update & Delete

- ◆ Update Example

```
UPDATE customers  
SET profession = 'Senior Engineer'  
WHERE cust_id = 104;
```

```
UPDATE customers_txn  
SET profession = 'Senior Engineer'  
WHERE cust_id = 4000005;
```

- ◆ Delete Example

```
DELETE FROM customers_txn  
WHERE age < 25;
```


◆ Insert Example

```
INSERT INTO customers_txn VALUES ( 4000005, 'Karen', 'Kale', 28, 'Analyst');
```

Check Results

```
SELECT * FROM customers_txn;
```

Compacting Transactions

Compaction merges delta files after updates/deletes for efficiency.

```
SET hive.compactor.initiator.on = true;
```

```
SET hive.compactor.worker.threads = 1;
```

```
ALTER TABLE customers_txn COMPACT 'major';  
SHOW COMPACTIONS;
```